Exact k-Dispersion via Fast-Matrix-Multiplication Based Clique Detection

Implementation and Verification of Section 3 of
Chen and Dumitrescu, *A Couple of Simple Algorithms for k-Dispersion*

Project Team
Major Project

October 27, 2026

Abstract

Given n points in $\mathbb{R}^d$ and an integer $k \leq n$, the k-dispersion problem asks for k of the points whose minimum pairwise distance is as large as possible. Chen and Dumitrescu [1] show (Theorem 2) that for $2 \leq k \leq 8$ the problem can be solved by reducing it to k-clique detection in threshold graphs and then to triangle detection, solved with fast rectangular matrix multiplication in the style of Nešetřil–Poljak [4] and Eisenbrand–Grandoni [3]. This report describes a C++ implementation of that algorithm: the notation, the algorithms, the implementation choices (exact integer arithmetic, bitset graphs, the per-k clique split taken from the paper's proof of Theorem 2, and an isolated matrix-multiplication backend), and a cross-verification against an independent exhaustive solver on a deterministic dataset of 288 instances drawn from 7 point distributions. The FMM-based solver returned the same optimum as the exhaustive solver on 288/288 instances, and every returned point set was independently confirmed to achieve the reported optimum.

Contents

1 Introduction **3**

2 Notation **3**

3 Background from the paper **3**
3.1 From dispersion to cliques 3
3.2 From cliques to triangles 4
3.3 The splits used in Theorem 2 4

4 Algorithms implemented **5**
4.1 Algorithm 1: k-dispersion by binary search 5
4.2 Algorithm 2: FMM-based k-clique detection 6
4.3 Algorithm 3: clique enumeration 6
4.4 Algorithm 4: exhaustive oracle 6

5 Implementation **7**
5.1 Code organisation 7
5.2 Design decisions 7
5.3 Complexity of the implementation 8
5.4 Usage 8

6 Verification methodology **8**
6.1 Approach 8
6.2 Dataset 8
6.3 Reproducing 9

7 Results **9**
7.1 Correctness 9
7.2 Running time 10

8 Limitations and future work **11**

9 Conclusion **11**

1 Introduction

The *(max-min) k-dispersion problem* asks to place k facilities at k of n candidate locations so that the facilities are as far apart as possible, measured by the smallest pairwise distance. It is NP-hard already in the plane when k is part of the input [7], so research focuses on running times of the form $n^{f(k)}$ for fixed k.

The paper of Chen and Dumitrescu [1] contributes three kinds of algorithms:

1. a recursive *combinatorial* algorithm running in $O(n^{k-1} \log n)$ time in the plane (Theorem 1) and a similar bound in $\mathbb{R}^3$ (Theorem 3);
2. *FMM based* algorithms (Section 3, Theorem 2) that improve the earlier bounds of Akagi et al. [2] for $k = 4, 5, 7, 8$ by choosing suitable *rectangular* matrix products;
3. a linear-time 0.99-approximation for random points (Theorem 4).

Scope of this project. This project implements the FMM based algorithm of item 2 (Section 3 / Theorem 2 of [1]), which works for points in any dimension d and, more generally, for any complete graph with positive edge weights. We additionally implement an exhaustive solver that is used purely as a correctness oracle. The combinatorial algorithm $A(k)$ and the approximation algorithm $B(k)$ are out of scope.

Contributions.

- An exact solver for k-dispersion with integer coordinates in any dimension, built on the triangle-detection reduction with the clique split (k_1, k_2, k_3) prescribed by the proof of Theorem 2.
- An instrumented driver reporting binary-search steps, clique-list sizes and solve time.
- An independent exhaustive oracle, a reproducible dataset of 288 instances, and a verification harness that checks every answer.

2 Notation

Table 1 lists the symbols used throughout the report. The same names appear in the source code wherever possible.

We work exclusively with *squared* distances. Since $x \mapsto x^2$ is strictly increasing on $[0, \infty)$, maximising the minimum distance and maximising the minimum squared distance select the same sets, and squared distances of integer points are integers, so all comparisons are exact.

3 Background from the paper

3.1 From dispersion to cliques

Lemma 1. $\mathrm{OPT} \in \mathcal{R}$, *i.e. the optimum is one of the pairwise (squared) distances.*

Proof. $\delta(S)$ is a minimum over pairs of S, hence equals $D(i, j)$ for some pair; in particular for an optimal S. □

Lemma 2. *For every r, there is S with $|S| = k$ and $\delta(S) \geq r$ if and only if G_r contains a K_k. Moreover, feasibility is monotone: if G_r contains a K_k and $r' \leq r$, then so does $G_{r'}$.*

Proof. $\delta(S) \geq r$ means every pair of S is at squared distance $\geq r$, i.e. every pair is an edge of G_r. Monotonicity follows from $E(G_r) \subseteq E(G_{r'})$. □

Table 1: Notation.

Symbol	Meaning	In code
$P = \{p_0, \dots, p_{n-1}\}$	input point set, $p_i \in \mathbb{Z}^d$	`points_`
$n,\ d,\ k$	number of points, dimension, number of points to select	`n_, d_, k_`
$D(i,j)$	squared Euclidean distance $\|p_i - p_j\|^2$	`dist2_[i][j]`
$S \subseteq P$, $\|S\| = k$	a candidate solution	
$\delta(S)$	$\min_{p \neq q \in S} \|p - q\|^2$, value of S	
OPT	$\max_{\|S\|=k} \delta(S)$, the optimum (squared)	`squared_distance`
$\mathcal{R}$	sorted set of distinct values $D(i,j)$, $i < j$	`candidate_thresholds_`
G_r	threshold graph on P: $\{i,j\} \in E \iff D(i,j) \geq r$	`ThresholdGraph`
K_k	complete graph on k vertices (k-clique)	
(k_1, k_2, k_3)	clique split, $k_1 + k_2 + k_3 = k$	`CliqueSplit`
$\mathcal{C}_q(G)$	set of all q-cliques of G	`enumerate_cliques`
V_1, V_2, V_3	$\mathcal{C}_{k_1}(G), \mathcal{C}_{k_2}(G), \mathcal{C}_{k_3}(G)$	`V1, V2, V3`
$A \sim B$	$A \cup B$ is a clique of G (implies $A \cap B = \emptyset$)	`compatible`
M_{12}, M_{23}, M_{13}	0/1 compatibility matrices between V_a and V_b	`M12, M23, M13`
$\Pi = M_{12}M_{23}$	integer product; $\Pi[a,c]$ counts 2-paths	`P`
ω	matrix multiplication exponent, $\omega < 2.372$ [6]	
$\omega(p,q,r)$	exponent for multiplying an $n^p \times n^q$ by an $n^q \times n^r$ matrix	
$e(k)$	exponent of the running time in Theorem 2 of [1]	

By Lemmas 1 and 2, OPT is the largest $r \in \mathcal{R}$ for which G_r contains a K_k, which can be found by binary search over $\mathcal{R}$ with $\lceil \log_2(|\mathcal{R}|+1) \rceil = O(\log n)$ clique tests. This is the framework of Akagi et al. [2], which [1] builds on.

3.2 From cliques to triangles

Let $k = k_1 + k_2 + k_3$ with $k_i \geq 0$. Build the tripartite auxiliary graph H whose parts are V_1, V_2, V_3 (the k_i-cliques of G), with an edge between clique-nodes $A \in V_a$ and $B \in V_b$ ($a \neq b$) iff $A \sim B$.

Lemma 3 (Nešetřil–Poljak [4])**.** *G contains a K_k if and only if H contains a triangle (A_1, A_2, A_3) with $A_i \in V_i$. In that case $A_1 \cup A_2 \cup A_3$ is a K_k.*

Proof. ($\Leftarrow$) Pairwise compatibility means every pair of vertices from different A_i is an edge; within each A_i every pair is an edge since A_i is a clique. Compatibility forces the A_i to be disjoint (a vertex is not adjacent to itself), so the union has exactly k vertices. ($\Rightarrow$) Split the vertices of a K_k into parts of sizes k_1, k_2, k_3. □

A triangle exists iff there are a, c with $M_{13}[a,c] = 1$ and $(M_{12}M_{23})[a,c] > 0$. Computing $\Pi = M_{12}M_{23}$ is a product of an $|V_1| \times |V_2|$ by a $|V_2| \times |V_3|$ matrix, i.e. of shape $n^{k_1} \times n^{k_2}$ by $n^{k_2} \times n^{k_3}$ in the worst case, which costs $O(n^{\omega(k_1,k_2,k_3)})$ with fast rectangular matrix multiplication.

3.3 The splits used in Theorem 2

Nešetřil and Poljak use the balanced split; the improvement of [1] comes from using the rectangular product that is cheapest for each k, following Eisenbrand and Grandoni [3]. Table 2 reproduces the exponents stated in the proof of Theorem 2 and the split that realises each one. These are the splits used by our implementation (function `clique_split`).

For $k \geq 9$ the paper gives no instantiation; we fall back to the balanced split of Nešetřil and Poljak. For $k = 2$ the empty clique forms V_1 and the triangle test degenerates into an edge test.

Table 2: Exponents $e(k)$ from the proof of Theorem 2 in [1], compared with the earlier bound $\omega\lfloor k/3\rfloor + (k \bmod 3)$ of [2] (both without the $\log n$ factor), and the clique split implemented.

k	$e(k)$ in [1]	bound	previous [2]	split (k_1, k_2, k_3)
2	(diameter)	–	–	$(0, 1, 1)$
3	(triangle)	ω	–	$(1, 1, 1)$
4	$\omega(1, 1, 2)$	< 3.251	3.372	$(1, 1, 2)$
5	$\omega(2, 1, 2) = 2\omega(1, 0.5, 1)$	< 4.086	4.372	$(2, 1, 2)$
6	2ω	< 4.744	4.744	$(2, 2, 2)$
7	$\omega(2, 3, 2) = 2\omega(1, 1.5, 1)$	< 5.590	5.744	$(2, 3, 2)$
8	$2\omega(1, 1, 2) = \omega(2, 2, 4)$	< 6.502	6.744	$(2, 2, 4)$
≥ 9	not covered	–	–	$(\lfloor \frac{k}{3} \rfloor, \lceil \frac{k-1}{3} \rceil, \lceil \frac{k}{3} \rceil)$

4 Algorithms implemented

4.1 Algorithm 1: k-dispersion by binary search

Algorithm 1 DISPERSION(P, k) (`FMMKDispersion::solve`)

1: compute $D(i, j)$ for all $i < j$ ▷ exact, 128-bit
2: $\mathcal{R} \leftarrow$ sorted distinct values of D
3: $lo \leftarrow 0$, $hi \leftarrow |\mathcal{R}| - 1$, $best \leftarrow \perp$
4: **while** $lo \leq hi$ **do**
5: $\quad mid \leftarrow \lfloor (lo + hi)/2 \rfloor$, $r \leftarrow \mathcal{R}[mid]$
6: $\quad$ build G_r ▷ bitset adjacency
7: $\quad (found, W) \leftarrow$ FMMCLIQUE(G_r, k)
8: $\quad$ **if** $found$ **then** $best \leftarrow (r, W)$; $lo \leftarrow mid + 1$
9: $\quad$ **else** $hi \leftarrow mid - 1$
10: $\quad$ **end if**
11: **end while**
12: **return** $best$ ▷ OPT $= r$, W is an optimal k-set

The smallest value of $\mathcal{R}$ is always feasible (G_r is complete), so $best \neq \perp$ whenever $n \geq k \geq 2$.

4.2 Algorithm 2: FMM-based k-clique detection

Algorithm 2 FMMClique(G, k) (`FMMKDispersion::fmm_clique`)

$(k_1, k_2, k_3) \leftarrow$ CliqueSplit(k) ▷ Table 2
$V_i \leftarrow$ EnumerateCliques(G, k_i) for $i = 1, 2, 3$
if some $V_i = \emptyset$ **then return** (**false**, –)
end if
$M_{12}[a, b] \leftarrow [V_1[a] \sim V_2[b]]$
$M_{23}[b, c] \leftarrow [V_2[b] \sim V_3[c]]$
$M_{13}[a, c] \leftarrow [V_1[a] \sim V_3[c]]$
$\Pi \leftarrow M_{12} \cdot M_{23}$ ▷ the matrix multiplication step
for each (a, c) with $M_{13}[a, c] = 1$ and $\Pi[a, c] > 0$ **do**
    find b with $M_{12}[a, b] = M_{23}[b, c] = 1$ ▷ witness recovery
    return (**true**, $V_1[a] \cup V_2[b] \cup V_3[c]$)
end for
return (**false**, –)

Correctness follows from Lemma 3. Witness recovery scans one row/column pair, $O(|V_2|)$, once; it is needed only to output the selected points, not to decide feasibility.

4.3 Algorithm 3: clique enumeration

EnumerateCliques(G, q) (`clique.cpp`) is a depth-first search that extends a partial clique C only with candidate vertices of larger index that are adjacent to every vertex of C. The candidate list is filtered on each extension, so every q-clique is produced exactly once, in increasing vertex order. For $q = 0$ it returns the single empty clique.

4.4 Algorithm 4: exhaustive oracle

`verify/brute_force.cpp` enumerates all $\binom{n}{k}$ subsets in lexicographic order and keeps the first one of maximum $\delta(S)$. It reuses prefix minima $\mathrm{cur}[i] = \delta(\{p_{\mathrm{idx}[0]}, \ldots, p_{\mathrm{idx}[i]}\})$ so that advancing to the next subset only recomputes the changed suffix. It shares no code with the FMM solver and runs in $O(\binom{n}{k}k^2)$ time, which is fast enough for the instance sizes used in verification.

5 Implementation

5.1 Code organisation

File	Content
`dispersion.hpp/.cpp`	`FMMKDispersion`: distances, candidate set $\mathcal{R}$, threshold graphs, `clique_split`, Algorithm 2, binary search (Algorithm 1), instrumentation (`SolveStats`).
`clique.hpp/.cpp`	`ThresholdGraph` (bitset adjacency), `enumerate_cliques`, `compatible`.
`matrix.hpp/.cpp`	dense row-major `Matrix` and the multiplication backend `multiply_blocked`.
`main.cpp`	command-line driver; `--stats` prints the instrumentation to `stderr`.
`verify/brute_force.cpp`	exhaustive oracle (Algorithm 4).
`verify/generate_dataset.py`	deterministic dataset generator.
`verify/run_verification.py`	cross-verification and timing harness; writes `verify/results/*.csv` and the tables in this report.

5.2 Design decisions

Exact arithmetic. Coordinates are signed 64-bit integers and squared distances are stored as `unsigned __int128`. No square roots or floating point are used anywhere, so ties (common on grids) are resolved exactly. The result is exact as long as $d \cdot (2 \max |x|)^2 < 2^{128}$; e.g. $|x| < 2^{61}$ for $d \leq 16$.

Threshold graphs as bitsets. Row i of the adjacency matrix of G_r is stored as $\lceil n/64 \rceil$ 64-bit words, so each edge query in clique enumeration and in `compatible` is a shift and a mask.

Compatibility. $A \sim B$ is tested by checking all $|A| \cdot |B|$ cross pairs. Because `edge(u,u)` is false, overlapping cliques are automatically incompatible, which enforces disjointness without an extra test.

Clique split. `clique_split(k)` returns exactly the splits of Table 2. The product computed is $\Pi = M_{12}M_{23}$, so its shape $|V_1| \times |V_2|$ by $|V_2| \times |V_3|$ matches the rectangular product $\omega(k_1, k_2, k_3)$ used in the paper's exponent.

Matrix-multiplication backend. All matrix multiplication goes through one function, `multiply_blocked`: an exact, cache-blocked (64×64 tiles) classical product over 64-bit unsigned integers that skips zero entries of the left operand. The paper's asymptotic bounds rely on fast rectangular matrix multiplication algorithms [3, 5, 6], which have very large constants and are not used in practice; the paper does not prescribe a concrete implementation. Isolating the product means a fast algorithm can be substituted without touching the rest of the code. Entries of Π are at most $|V_2| \leq \binom{n}{k_2}$, far below 2^{64}.

Instrumentation. `SolveStats` records $|\mathcal{R}|$, the number of binary-search steps (clique tests), the number of matrix products, and the largest $|V_1|, |V_2|, |V_3|$ seen. With `--stats` the driver prints these together with the split and the solve time.

5.3 Complexity of the implementation

Each clique test costs $O(n^{\max k_i} k)$ for enumeration, $O(|V_a||V_b|k_a k_b)$ to fill the three matrices, and the product. With the classical backend the product costs $O(|V_1||V_2||V_3|) = O(n^k)$, so the total is $O(n^k \log n)$. Replacing `multiply_blocked` by a fast rectangular algorithm gives the $O(n^{e(k)} \log n)$ bound of Table 2. The paper notes that the $\log n$ factor disappears once the exponents are rounded up. The memory used is dominated by the three matrices, $O(|V_1||V_2| + |V_2||V_3| + |V_1||V_3|)$ words.

5.4 Usage

```
g++ -O2 -std=c++17 *.cpp -o dispersion
./dispersion [--stats] < input.txt

# input:  n d k, then n lines of d integers
# output:
optimal_squared_distance 1261
selected_indices 2 8 11 13 19 23
selected_points
6 9
...
```

6 Verification methodology

6.1 Approach

The FMM solver is verified *differentially* against the exhaustive oracle of Algorithm 4. The two programs share no code (distance computation, search, and output are written independently) and read the same input files. For every instance the harness `verify/run_verification.py` checks:

1. **Optimum agreement:** both programs report the same OPT.
2. **Witness validity:** each returned index set has exactly k distinct indices in $[0, n)$.
3. **Witness value:** $\delta(S)$ of each returned set, recomputed in Python with arbitrary-precision integers, equals the reported optimum. This independently confirms the FMM solver's certificate.
4. **Split conformance:** the split reported by the solver equals the one in Table 2.

Whether the two programs return the *same* set is also recorded. This is informational only, since optimal sets need not be unique.

6.2 Dataset

The dataset is produced by `verify/generate_dataset.py` with a fixed seed and stored in `verify/dataset/` (one file per instance plus `manifest.csv`). For every family and dimension it contains $k = 2, \ldots, 9$ with three values of n each (smallest, middle, largest of the range $8 \le n \le 40$ for $k = 2$ down to $10 \le n \le 15$ for $k = 9$), 288 instances in total.

Table 3: Point families in the verification dataset.

Family	Construction and purpose	d
`uniform`	integer points uniform in $[0, 1000]^d$; generic case	1,2,3
`clustered`	Gaussian clusters ($\sigma = 40$) around 3–4 centres; skewed distance distribution	2,3
`grid`	random subset of an integer lattice; many tied distances	2,3
`duplicate`	points drawn with repetition from a small pool; coincident points ($D = 0$)	2
`collinear`	points on a line $y = ax + b$; degenerate geometry	2
`circle`	points near a circle of radius 1000; almost-equal distances	2
`large`	coordinates in $[-10^9, 10^9]$; squared distances beyond 2^{32}, exercising 128-bit arithmetic	2,3

6.3 Reproducing

```
python3 verify/generate_dataset.py      # (re)creates verify/dataset/
python3 verify/run_verification.py      # builds, verifies, times
```

The harness exits with a non-zero status if any check fails. It also writes the LaTeX fragments in `report/generated/` used by the tables and plots below.

7 Results

7.1 Correctness

Check (all 288 instances)	passed
Same optimum from FMM solver and oracle	**288/288**
FMM witness valid and achieves the optimum	**288/288**
Oracle witness valid and achieves the optimum	**288/288**
Identical selected sets (informational)	288/288

Table 4 breaks the results down by k, together with the solver statistics. Table 5 breaks them down by point family, and Table 6 lists individual instances side by side.

Table 4: Verification by k. "Tests" is the mean number of clique tests (binary-search steps); "max $|V|$" the largest clique list built; times are mean in-process solve times in milliseconds.

k	split	inst.	n	OPT match	FMM witness ok	tests	max $\lvert V\rvert$	FMM ms	oracle ms
2	(0,1,1)	36	8–40	36/36	36/36	7.2	40	0.2	0.0
3	(1,1,1)	36	8–35	36/36	36/36	6.5	35	0.3	0.1
4	(1,1,2)	36	8–28	36/36	36/36	6.2	285	0.9	0.2
5	(2,1,2)	36	8–24	36/36	36/36	5.9	208	2.6	0.3
6	(2,2,2)	36	8–20	36/36	36/36	5.7	168	3.6	0.3
7	(2,3,2)	36	9–18	36/36	36/36	5.7	605	7.0	0.3
8	(2,2,4)	36	10–16	36/36	36/36	5.8	1820	13.0	0.2
9	(3,3,3)	36	10–15	36/36	36/36	5.5	455	31.1	0.1

Observations.

Table 5: Verification by point family. $|\mathcal{R}|$ is the mean number of distinct pairwise distances; it is small for `grid` and `duplicate` because of ties.

Family	d	instances	OPT match	same set	mean $\|\mathcal{R}\|$
`uniform`	1,2,3	72	72/72	72/72	153
`clustered`	2,3	48	48/48	48/48	163
`grid`	2,3	48	48/48	48/48	15
`duplicate`	2	24	24/24	24/24	29
`collinear`	2	24	24/24	24/24	132
`circle`	2	24	24/24	24/24	164
`large`	2,3	48	48/48	48/48	164

Table 6: Sample instances (`uniform`, $d = 2$, largest n for each k): optimal squared distance reported by both solvers.

Instance	n	k	OPT (FMM)	OPT (oracle)	match	witness ok	tests
`uniform_d2_k2_n40`	40	2	1671925	1671925	✓	✓	10
`uniform_d2_k3_n35`	35	3	835873	835873	✓	✓	9
`uniform_d2_k4_n28`	28	4	464722	464722	✓	✓	8
`uniform_d2_k5_n24`	24	5	219088	219088	✓	✓	8
`uniform_d2_k6_n20`	20	6	140132	140132	✓	✓	7
`uniform_d2_k7_n18`	18	7	70538	70538	✓	✓	7
`uniform_d2_k8_n16`	16	8	52609	52609	✓	✓	7
`uniform_d2_k9_n15`	15	9	42233	42233	✓	✓	7

- No mismatches were found in any family, including the degenerate ones (coincident, collinear, tied, near-equal and very large values).
- The number of clique tests is $\lceil \log_2(|\mathcal{R}| + 1) \rceil$ or one less, as expected from binary search. It is about 5–10 even for $|\mathcal{R}|$ in the hundreds.
- The largest clique lists occur for the largest part, e.g. $\binom{16}{4} = 1820$ 4-cliques for $k = 8$ with split $(2, 2, 4)$. This shows how the split determines the matrix shapes.

7.2 Running time

Figure 1 shows the median (over three runs) in-process solve time for uniform planar points as n grows, for $k = 3, 4, 6$.

Discussion. With the classical backend the FMM-based solver is *not* faster than exhaustive search at these sizes, and for $k = 6$ it is several times slower. This is consistent with the analysis: each clique test performs about $|V_1||V_2||V_3| \approx n^k/(k_1!\, k_2!\, k_3!)$ operations, and there are $\Theta(\log n)$ tests. Exhaustive search performs about $n^k/k!$ subset steps. For $k = 6$, $k_1!k_2!k_3! = 8$ while $k! = 720$. The asymptotic gain of Theorem 2 comes entirely from replacing the classical product with fast rectangular matrix multiplication. The paper itself notes that FMM-based algorithms are generally considered impractical because of their large constants, and that it is unclear from which k they win in practice. The experiments confirm that the reduction is *correct*; demonstrating its *speed* would need a fast multiplication backend and much larger inputs.

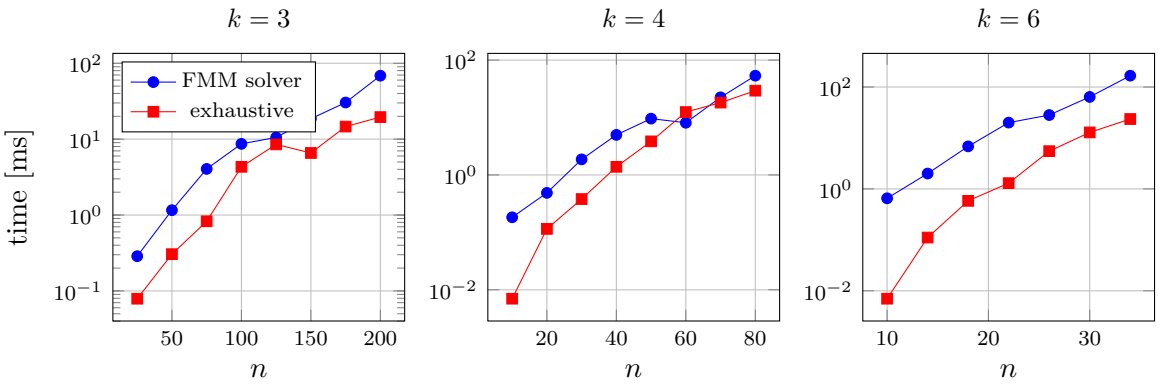


Figure 1: Solve time vs. n (log scale), uniform points in $[0, 10^6]^2$, classical matrix-multiplication backend. Optima agreed on every run.

8 Limitations and future work

- **Fast matrix multiplication.** The backend is classical; plugging in Strassen-like or rectangular algorithms, or a bit-packed Boolean product (only $\Pi[a, c] > 0$ is needed), is the natural next step.
- **Memory.** M_{13} and M_{23} have up to $\binom{n}{k_2}\binom{n}{k_3}$ entries stored as 64-bit words; bit-packing would cut memory by 64×.
- **Other algorithms of the paper.** The combinatorial algorithm $A(k)$ of Theorems 1 and 3 ($O(n^{k-1} \log n)$ in the plane) and the approximation algorithm $B(k)$ of Theorem 4 are not implemented.
- **Input range.** Exactness requires $d(2 \max |x|)^2 < 2^{128}$, see Section 5.2.

9 Conclusion

We implemented the FMM based exact algorithm for k-dispersion from Section 3 of Chen and Dumitrescu [1]: binary search over pairwise distances, reduction of each decision problem to k-clique detection in a threshold graph, and reduction of k-clique detection to triangle detection via a single matrix product with the clique split (k_1, k_2, k_3) prescribed by the proof of Theorem 2. A differential test against an independent exhaustive solver on 288 instances from 7 families (including degenerate inputs) found no discrepancy, and every returned solution was independently certified. The running-time experiments show that, as the authors anticipate, the asymptotic advantage depends on fast matrix multiplication and does not appear with a classical backend at practical sizes.

References

[1] K. Chen and A. Dumitrescu, *A Couple of Simple Algorithms for k-Dispersion*, arXiv:2511.00692, 2025.

[2] T. Akagi, T. Araki, T. Horiyama, S. Nakano, Y. Okamoto, Y. Otachi, T. Saitoh, R. Uehara, T. Uno, and K. Wasa, Exact algorithms for the max-min dispersion problem, *FAW 2018*, LNCS 10823, pp. 263–272, 2018.

[3] F. Eisenbrand and F. Grandoni, On the complexity of fixed parameter clique and dominating set, *Theoretical Computer Science* 326(1–3):57–67, 2004.

[4] J. Nešetřil and S. Poljak, On the complexity of the subgraph problem, *Commentationes Mathematicae Universitatis Carolinae* 26(2):415–419, 1985.

[5] F. Le Gall and F. Urrutia, Improved rectangular matrix multiplication using powers of the Coppersmith–Winograd tensor, *SODA 2018*, pp. 1029–1046.

[6] V. Vassilevska Williams, Y. Xu, Z. Xu, and R. Zhou, New bounds for matrix multiplication: from alpha to omega, *SODA 2024*, pp. 3792–3835.

[7] D.-W. Wang and Y.-S. Kuo, A study on two geometric location problems, *Information Processing Letters* 28(6):281–286, 1988.